

Pontificia Universidad
JAVERIANA
Colombia

Pontificia Universidad Javeriana

Facultad de Ingeniería

Introducción a sistemas Distribuidos

Ana Maria Murcia Gómez

Martín David Sanmiguel Delgado

Jhon sebastian Mejia

David Felipe Mannios

12 de Octubre. 2026

Bogotá - Colombia.

Github: https://github.com/Martinjaveriana/Taller-MPI

1. Introducción

El presente documento evalúa el rendimiento de la multiplicación de matrices cuadradas distribuidas con MPI (Message Passing Interface) sobre un clúster de cuatro nodos Linux. Para ello, se busca analizar el comportamiento del cluster al ejecutar diferentes algoritmos de multiplicación de matrices y posteriormente, establecer una correlación entre los tiempos de ejecución y el número de workers.

2. Descripción de los sistemas de cómputo

Para realizar el experimento se utilizaron 4 sistemas de cómputo linux con una misma arquitectura de hardware. La tabla 1 a continuación, se muestran los componentes de hardware y los roles de cada uno de los equipos utilizados:

ID	CPU	Nucleos Logicos	Frec. Max. (GHz)	RAM (GiB)	Rol
PC-1	Intel Xeon Gold 6240R (VMware)	4 (vCPU)	2.4	12	**Server**
PC-2	Intel Xeon Gold 6348 (VMware)	4 (vCPU)	2.6	12	**Worker**
PC-3	Intel Xeon CPU E5-2650 (VMware)	4 (vCPU)	2.2	12	**Worker**
PC-4	Intel Xeon Gold 6348 (VMware)	4 (vCPU)	2.6	12	**Worker**

Tabla 1. Configuración de los sistemas de cómputo que hacen parte del cluster

Adicionalmente hay que resaltar que todas las máquinas usan el sistema operativo Linux Ubuntu

3. Diseño experimental

Para evaluar el rendimiento de la multiplicación de matrices distribuida con MPI se realizaron tres experimentos sobre el mismo clúster de cuatro nodos descrito en la Tabla 1. El primero, o experimento principal, mide cómo cambia el tiempo de ejecución de los algoritmos FxC y FxT al repartir el cálculo entre 1, 2 y 3 trabajadores. El segundo, de carácter complementario, compara la ejecución de la multiplicación de matrices usando el protocolo MPI de manera distribuida. El tercer experimento, también complementario,

calcula el tiempo que se pierde en enviar datos y esperar entre procesos (overhead) [19], al realizar la multiplicación de matrices.

Para que los resultados de los tres experimentos fueran consistentes entre sí, todos comparten los mismos parámetros:

Variable	Experimento 1	Experimento 2	Experimento 3
Algoritmo	Filas por columnas (FxC) Filas por transpuestas (FxT)	Filas por columnas (FxC) Filas por transpuestas (FxT)	Filas por columnas (FxC) Filas por transpuestas (FxT)
Tamaño de las matrices	192 x 192 384 x 384 768 x 768 1536 x 1536 2304 x 2304 3072 x 3072	192 x 192 384 x 384 768 x 768 1536 x 1536 2304 x 2304 3072 x 3072	192 x 192 384 x 384 768 x 768 1536 x 1536 2304 x 2304 3072 x 3072
Número de trabajadores	1, 2, 3	1, 2, 3	1, 2, 3

Tabla 2. variables utilizadas durante el experimento

Para los experimentos se utiliza el esquema maestro-trabajador de elaboración propia, en donde el master divide la matriz en entre cada uno de los trabajadores, estos realizan los cálculos y al terminar cada trabajador envía su resultado para que finalmente el máster pueda unirlos y dar con la respuesta final

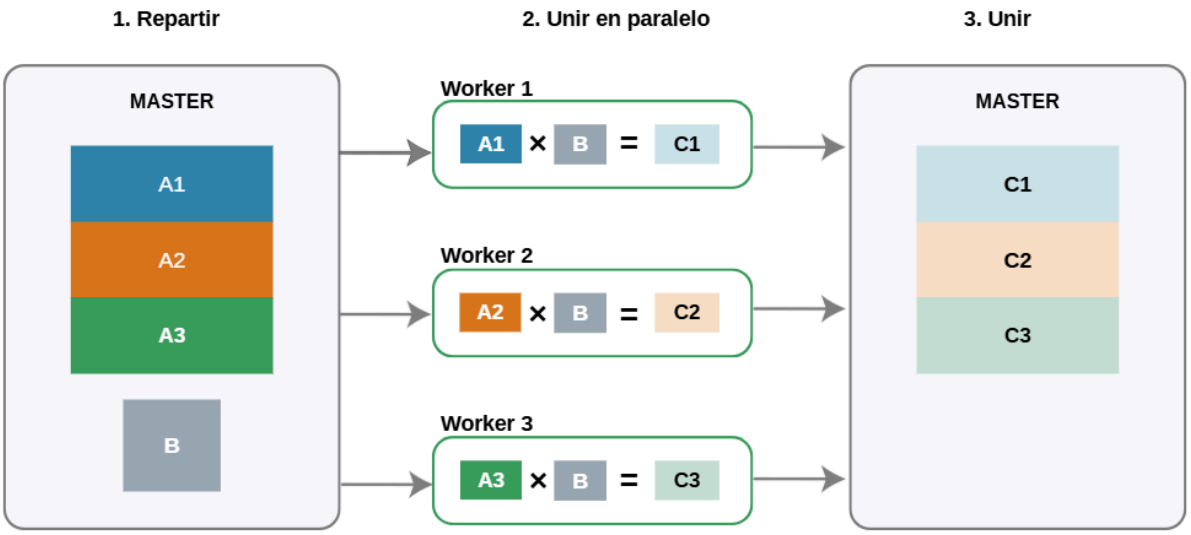


Figura 1. Imagen que explica la ejecución del MPI en el experimento.

3.1. Fuentes de ruido y número de repeticiones

En los sistemas linux y en los sistemas distribuidos hay muchos factores que pueden afectar el tiempo de ejecución de los algoritmos. Estas fuentes de variación se agrupan en 3 categorías:

a) **Variaciones por el sistema operativo:** El factor principal que puede generar alteraciones en el tiempo de ejecución es el ruido computacional (OS Jitter) [1]. El OS Jitter ocurre porque, en los sistemas operativos basados en Linux, el Kernel gestiona interrupciones de hardware y procesos en segundo plano que pueden generar alteraciones en los tiempos de procesamiento al desviar recursos de CPU al programa de forma impredecible. En un trabajo paralelo con MPI este ruido se amplifica: si cualquiera de los trabajadores sufre una interrupción, el master debe esperarlo. El retraso no queda aislado, porque al final de la multiplicación el master necesita recibir las porciones de C de todos los trabajadores, de modo que el tiempo total queda marcado por el trabajador más lento. A esto se suma que el planificador puede migrar procesos entre núcleos, lo que obliga a recargar la memoria caché e incrementa los tiempos, especialmente en algoritmos sensibles a la caché como FxC [5].

b) **Variaciones originadas por MPI:** El arranque de los procesos mediante mpirun requiere conexiones SSH a cada nodo, lo que añade variabilidad al tiempo, sin embargo, este costo no afecta las mediciones, porque el tiempo se empieza a medir cuando todos los procesos ya existen y el master comienza a enviar los datos [0].

c) **Variaciones originadas por la red y el hardware:** La red del laboratorio es compartida, por lo que el tráfico de otros equipos puede reducir el ancho de banda

disponible durante la ejecución de los experimentos. Además, los cuatro nodos son máquinas virtuales, haciendo que ejecuciones de otras máquinas virtuales o procesos dentro del servidor puedan afectar los tiempos de ejecución.

Por todo lo anterior se decidió que para mitigar el efecto de estas fuentes de variación se aplicó el principio de la Ley de los Grandes Números [2], en los tres experimentos, cada combinación de algoritmo, tamaño de matriz y número de trabajadores se ejecutó 30 veces. Finalmente, al promediar esta muestra, se logra aislar el tiempo real de cálculo matemático, garantizando así la validez y precisión de los datos analizados.

Adicionalmente para la ejecución los experimentos se utilizaron los siguientes algoritmos:

- **Algoritmo 1:** Consiste en el método tradicional de multiplicar matrices, el cual opera multiplicando cada elemento de la fila de la primera matriz por cada elemento de la columna de la segunda matriz. Durante la evaluación de rendimiento, este método fue denominado como "Filas por columnas (FxC)". Al analizar este algoritmo desde la óptica del rendimiento, se concluye que es computacionalmente ineficiente debido a la forma en que el lenguaje C estructura los datos en la memoria, utilizando el principio conocido como Row-Major Order o almacenamiento por filas [3] [4]. Al recorrer la segunda matriz por columnas, el sistema operativo consume más tiempo adquiriendo la información, ya que tiene que hacer recorridos adicionales a través de la jerarquía de memoria al tener que traer datos de la memoria principal, violando el principio de localidad espacial y por ende, se consume más tiempo realizando la operación. En la figura 1 se puede observar cómo es que realmente los datos de los arreglos multidimensionales son almacenados. Así que se pueden esperar mayores tiempos de procesamiento para este algoritmo.

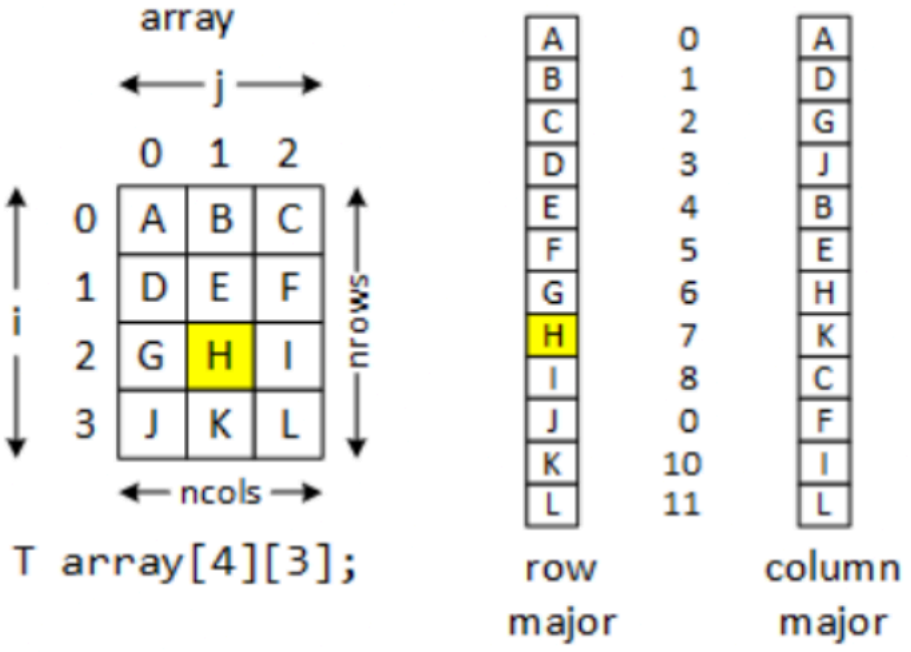


Figura 2: imagen tomada de [5] que representa Row-Major Order o almacenamiento por filas demostrando cómo el lenguaje C, almacena los datos de los arreglos multidimensionales

- **Algoritmo 2:** El algoritmo es una variante en donde representa una optimización del primer algoritmo en donde se tiene en cuenta la jerarquía de memoria del computador. Durante la evaluación, este método será referido como "Filas por transpuestas (FxT)". Este algoritmo maximiza el uso del principio de localidad espacial ya que, al transponer primero la segunda matriz, sus columnas se convierten en filas y se garantiza que la información requerida por el procesador se encuentre en posiciones de memoria contiguas. De esta manera, se minimizan las consultas del sistema operativo a la memoria principal, así que se espera un tiempo significativamente menor en comparación con el algoritmo Fx

Utilizando esos algoritmos

3.2. **Experimento 1:** El experimento principal evalúa el tiempo de ejecución de dos algoritmos de multiplicación de matrices en un entorno distribuido con distintos números de workers. Ambos algoritmos se ejecutan con el mismo programa MPI, la única diferencia es que en FxT el master transpone la matriz antes de enviarla.

3.3. **Experimento 2:** En este experimento se busca responder si con los mismos procesos MPI, conviene más ejecutar de manera distribuida (en el clúster) o en de manera local (solo un nodo o sistema de computo). Para ello se comparan dos escenarios:

- **Distribuida:** el master y los trabajadores se ejecutan en sistemas de cómputo distintos y se comunican a través de la red.
- **Local:** todos los procesos (el master y los 3 trabajadores) se ejecutan dentro de un solo computador

3.4. **Experimento 3:** Por último se midió la fracción del tiempo de una multiplicación con MPI que es el overhead. El overhead es el tiempo que se invierte en realizar las tareas administrativas necesarias para realizar el cálculo como enviar y recibir mensajes, también se podría definir como el tiempo **(**todo el tiempo invertido en tareas distintas al cálculo**) [**19**]**. Se realizan las siguientes mediciones:

- **Tiempo total:** Para esto se mide el tiempo total de la multiplicación de matrices, iniciando desde el envío de B hasta que se recibe la última parte de C, la comunicación entre el server y los workers, el tiempo de multiplicación y la creación del resultado final.
- **Tiempo de cálculo:** Se midió el tiempo de cálculo de cada uno de los workers, y se elige el mayor tiempo, porque este es el tiempo que el master tiene que esperar al trabajador más lento. Así que las fórmulas que usamos fueron las siguientes [19]:

$$Overhead = Tiempo\ total\ de\ ejecución - Tiempo\ de\ cálculo$$

$$\%\ de\ Overhead = \frac{Tiempo\ total - Tiempo\ de\ cálculo}{Tiempo\ total}$$

4. Recolección y procesamiento de los datos

Dado que los sistemas operativos de propósito general ejecutan múltiples procesos en segundo plano que pueden alterar el rendimiento de la CPU en un momento dado, por lo tanto, se realizó el experimento varias veces para mitigar este tipo de variaciones.

Para garantizar la integridad y repetibilidad de los experimento, la recolección se dividió en dos fases automatizadas:

Fase 1. Ejecución: cada uno de los experimentos utiliza un script (.pl/.sh) para automatizar la ejecución de cada una de las pruebas. Los scripts se encargaron de ejecutar cada una de las variables (algoritmo, tamaño de matriz, número de workers) 30 veces consecutivas. posteriormente los datos individuales de cada iteración se exportaron a archivos de texto plano (.txt/.csv), generando un gran volumen de datos crudos

Fase 2. Consolidación y visualización: para evitar errores de manipulación manual humana, se desarrollo un script de procesamiento de datos en Python utilizando librerías de analisis de datos como matplotlib.

5. Análisis de resultados

5.1. Experimento 1

5.1.1. FxC vs FxT

N	W	FxC (s)	FxT (s) FxT
192	1	0,035	0,032
192	2	0,031	0,028
192	3	0,026	0,025
384	1	0,254	0,221
384	2	0,181	0,128
384	3	0,136	0,101
768	1	3,30	1,70
768	2	1,76	0,915
768	3	1,23	0,644
1536	1	40,5	13,8
1536	2	20,5	7,20
1536	3	13,1	4,91
2304	1	161,1	46,5
2304	2	80,6	24,0
2304	3	54,2	16,2
3072	1	451,2	110,2
3072	2	252,6	56,8
3072	3	164,2	38,1

Tabla 3. tabla en se muestra los tiempos de ejecución de los algoritmos FxC y FxT por cada número de workers y por cada tamaño de matriz

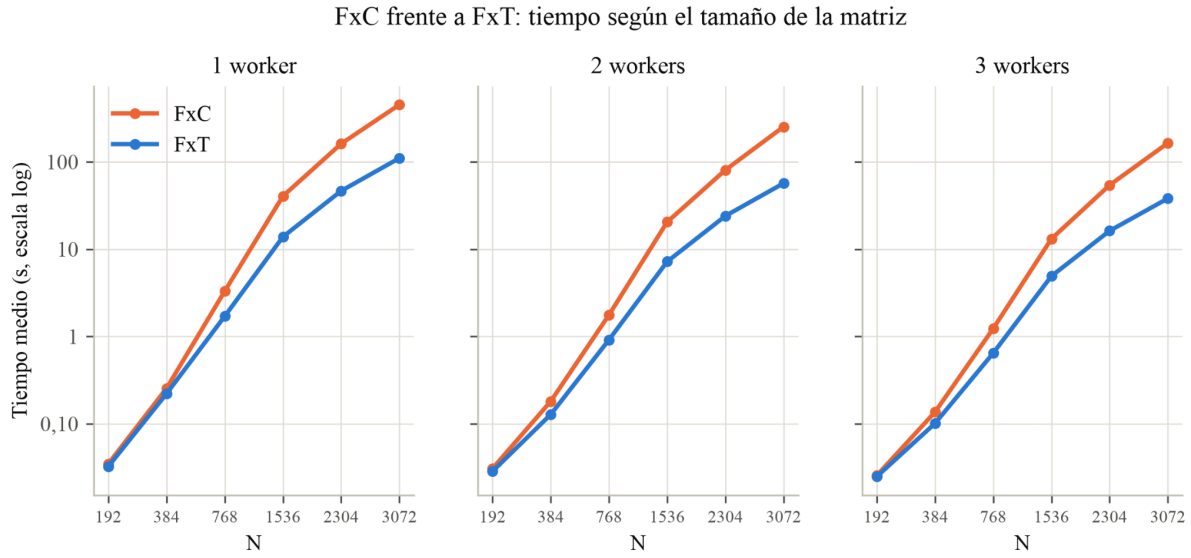


Figura 3. Se puede ver las gráficas en donde el eje "Y" es el tiempo de ejecución en una escala logarítmica y el eje "X" es el tamaño de las matrices a multiplicar. Se puede observar que se realizo una grafica por cada número de workers

al observar la Figura y la Tabla 3 podemos realizar las siguientes observaciones:

- **FxT es superior a FxC:** El algoritmo que transpone la matriz supera en tiempo en tiempo en todos los casos a la multiplicación de matrices tradicional, se puede resaltar que las mejoras más significativas se pueden observar al multiplicar matrices muy grandes, por ejemplo en la matriz de 3072 x 3072 hay mejoras de hasta un 400%
- **Gran diferencia de tiempos entre los tamaños de las matrices:** se realizó una gráfica con una escala logarítmica ya que puede observar que los tiempos de la multiplicación de matrices escalan muy rápido, se puede observar con 1 worker con el algoritmo de FxT, que al pasar de la matriz de 1536 a 3072 el tiempo pasa de 13,8s a 110,2s un aumento de aproximadamente del 800%

5.1.2. mejora al agregar un worker y eficiencia

FxC: tiempo real frente al esperado (mitad con 2 workers, un tercio con 3)

N	1 worker (s)	2 workers real (s)	2 workers esperado (s)	Eficiencia 2W	3 workers real (s)	3 workers esperado (s)	Eficiencia 3W
192	0,035	0,031	0,017	56,5 %	0,026	0,012	45,3 %
384	0,254	0,181	0,127	70,3 %	0,136	0,085	62,2 %
768	3,30	1,76	1,65	93,6 %	1,23	1,10	89,3 %
1536	40,5	20,5	20,2	98,8 %	13,1	13,5	102,8 %
2304	161,1	80,6	80,5	99,9 %	54,2	53,7	99,1 %
3072	451,2	252,6	225,6	89,3 %	164,2	150,4	91,6 %

Tabla 4. Tabla en donde se muestran los tiempos esperados y los tiempos reales entre el número de workers y los tamaños de matrices

Al observar, la tabla 4 se puede concluir que:

- **La mejora en las matrices pequeñas (N = 192 y 384) no es la esperada.** los tiempos se ven afectados por el overhead y por el tiempo que se gasta al enviar las matrices por la red, esto genera que en el caso de la matriz de 192 x 192 se vea una mejora de apenas 0.009s contra las 0.023s esperados al comparar el tiempo distribuir con 3 workers contra el tiempo de una ejecución en un solo nodo, ya que se supondría que la mejora esperada debería ser de una ejecución de un tercio del tiempo original

5.1.3. Desde qué tamaño distribuir

Utilizando la figura 4 se puede observar que en multiplicaciones de matrices demasiado pequeñas como “192 x 192” y “384 x 384” no se obtiene la ganancia de eficiencia esperada, mientras que en tamaños más grandes como en 768 hasta 3072 se obtienen ganancias de rendimiento superiores al 89%

5.2. Experimento 2

5.2.1. Local vs clúster según el algoritmo

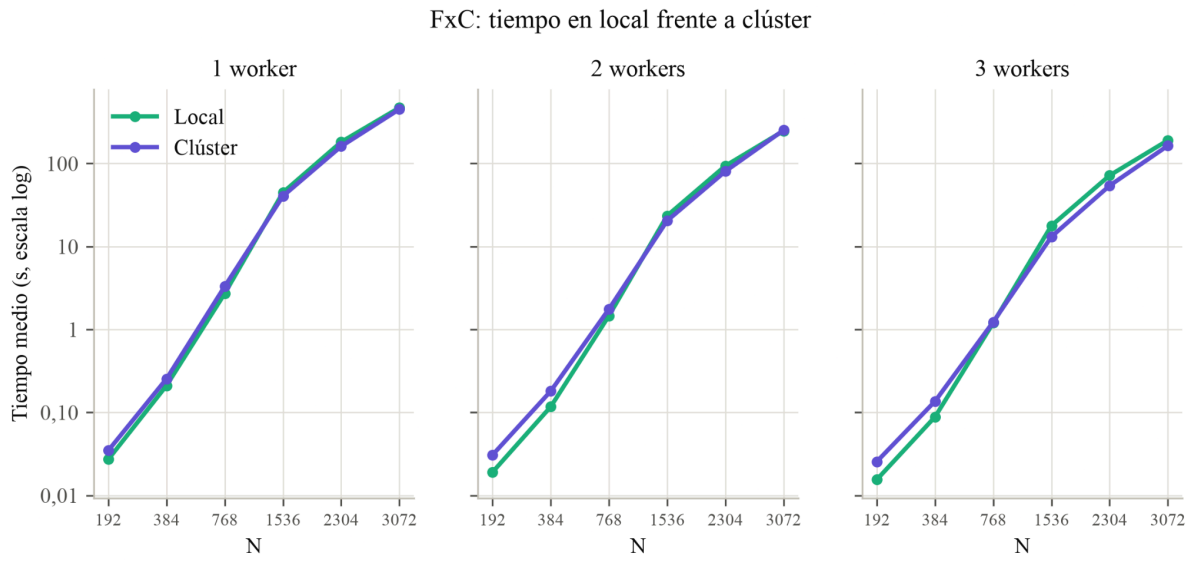


Figura 4. Se puede ver las gráficas en donde el eje "Y" es el tiempo de ejecución en una escala logarítmica y el eje "X" es el tamaño de las matrices a multiplicar. Se puede observar que se realizo una grafica por cada número de workers y que la línea morada representa los datos del cluster y la línea verde lo datos de un sistema de cómputo que ejecuta los cálculos de manera local del algoritmo de multiplicación de matrices

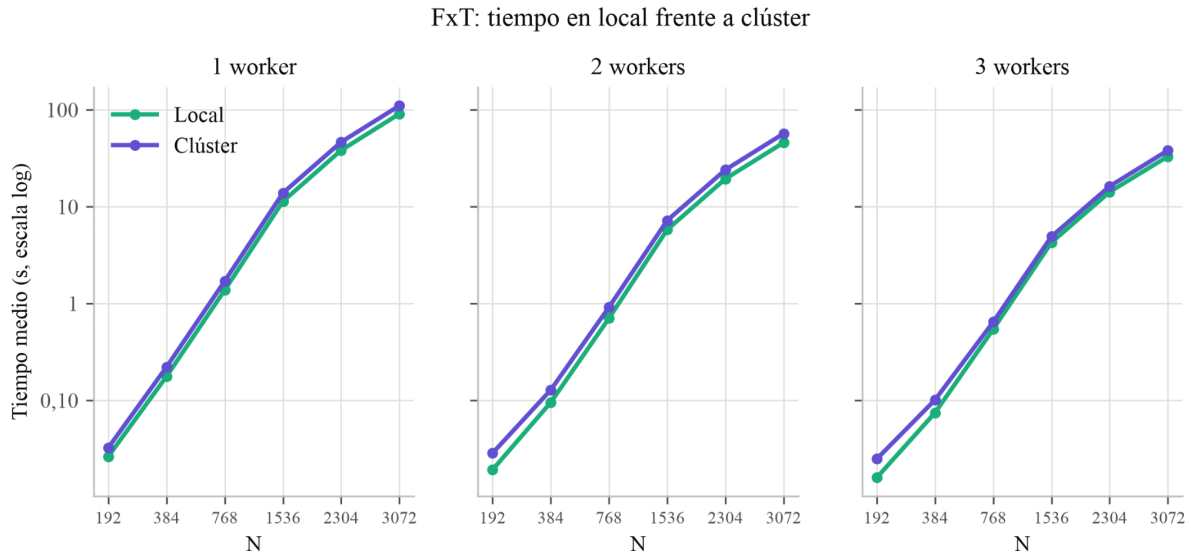


Figura 5. Se puede ver las gráficas en donde el eje "Y" es el tiempo de ejecución en una escala logarítmica y el eje "X" es el tamaño de las matrices a multiplicar. Se puede observar que se realizo una grafica por cada número de workers y que la línea morada representa los datos del cluster y la línea verde lo datos de un sistema de cómputo que ejecuta los cálculos de manera local del algoritmo de multiplicación de matrices

5.2.2. Escalabilidad

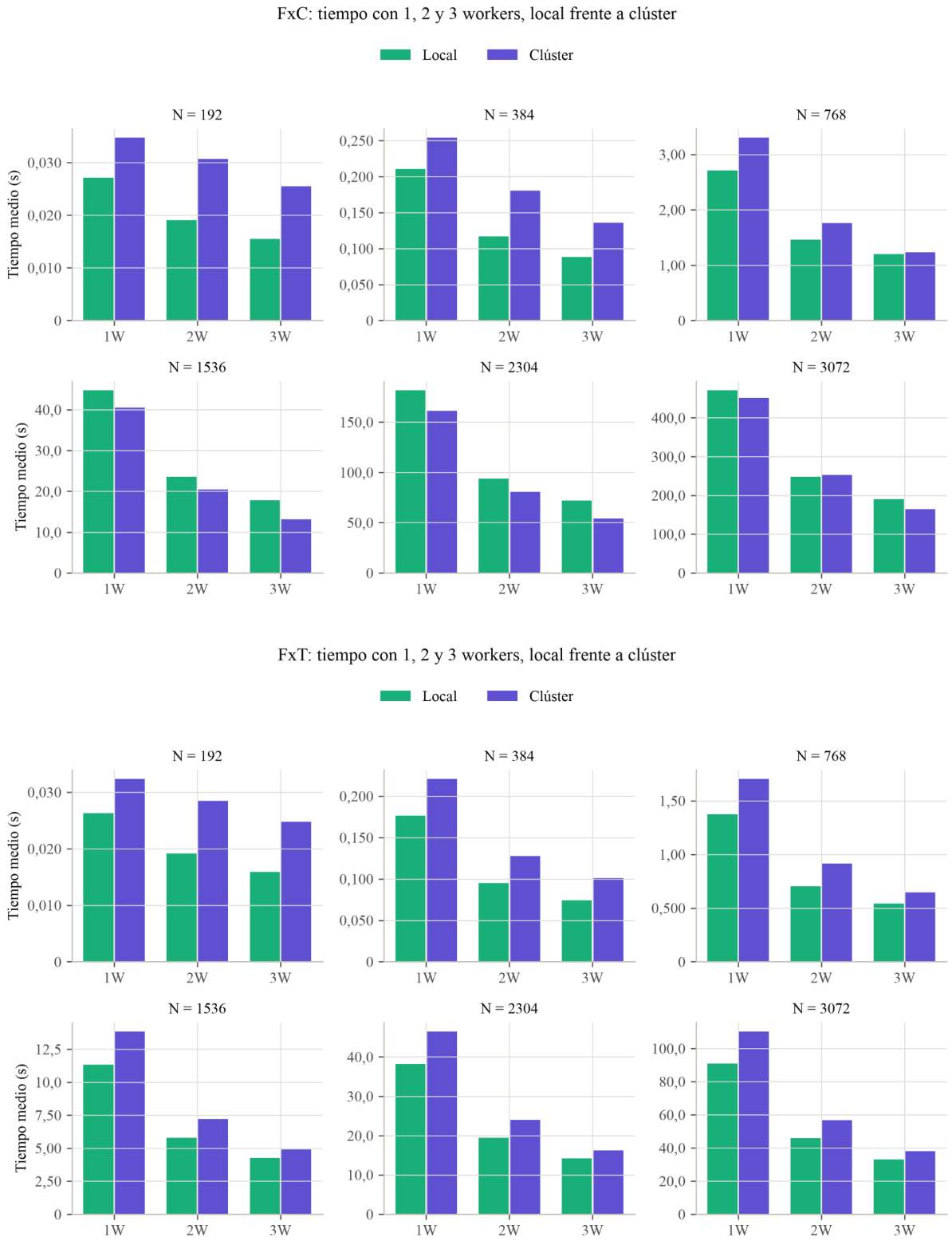


Figura 6 y 7. tiempos de ejecución cluster vs local en los algoritmos de FxC y FxT

Se puede observar que la ejecución local siempre gana cuando se trata de valores pequeños de matrices, esto puede generarse por el overhead y cuando las matrices son más grandes la

ejecución distribuida gana para FxC ya que ahí el overhead se vuelve una cantidad insignificante de tiempo. En FxT gana local en todos los tamaños.

5.3. Experimento 3

5.3.1. Overhead por N y workers

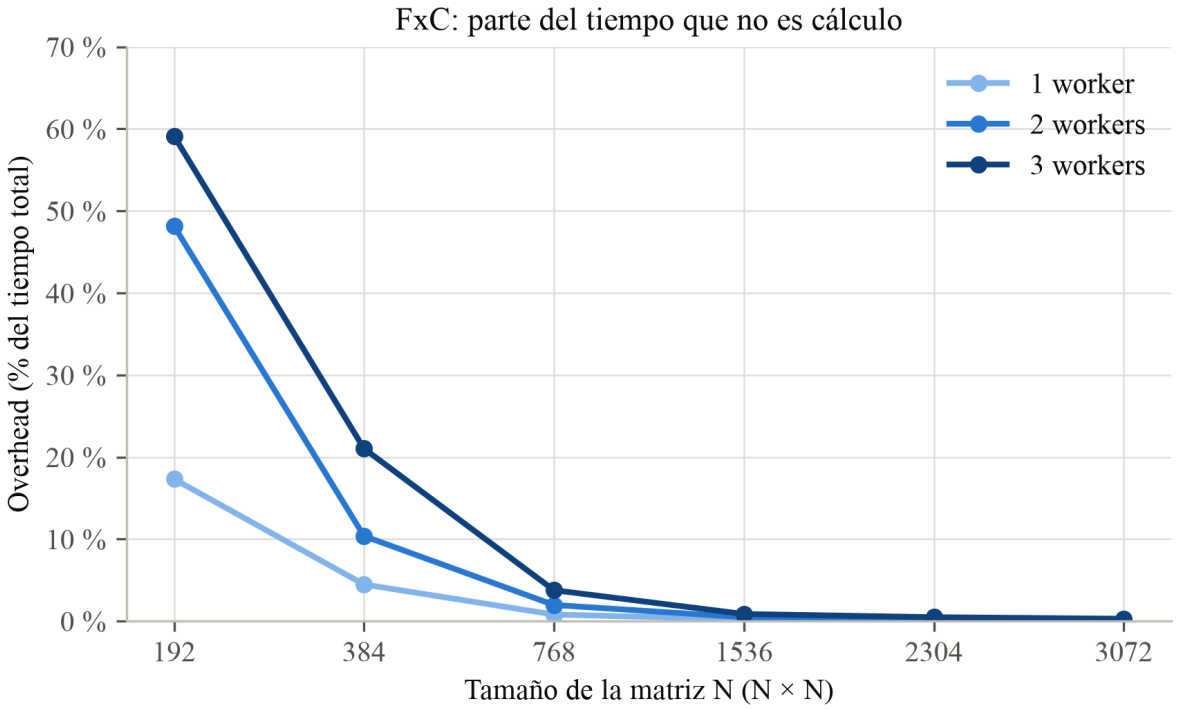


Figura 8. comparación del tiempo porcentual del overhead sobre el tiempo total por el número de workers y tamaños de matrices

se puede al observar la figura 8 se puede afirmar que:

- **El overhead en las matrices pequeñas es muy grande:** se puede observar que para matrices pequeñas el overhead es muy grande ya que consume una mayor parte del tiempo total, además se puede observar que la tarea de distribuir en matrices pequeñas no vale la pena ya que aumenta mucho en overhead de manera porcentual
- **El overhead disminuye muy rápido al aumentar el tamaño de las matrices:** se puede observar que el overhead pasa a tener un porcentaje muy pequeño del tiempo de ejecución cuando se crece el tamaño de las matrices
- **Distribuir entre más workers siempre significa aumentar el tiempo de overhead**
- **Con grandes tamaños de matrices el overhead deja de importar**: se puede observar que con grandes tamaños de matrices como el de 3072 x 3072 el overhead pasa a ser una cantidad demasiado insignificante

5.3.2. FxT vs FxC

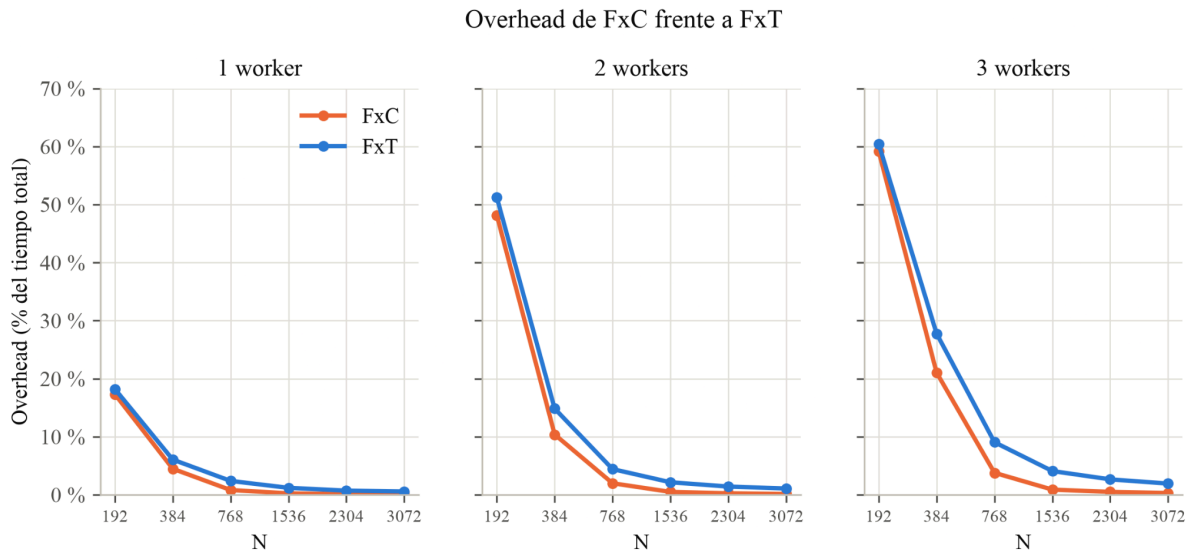


Figura 9. Gráficas que muestran el porcentaje de overhead para distintos workers con los distintos algoritmos

observando la figura 9 se puede afirmar que cuando los tamaños de la matriz son pequeños el overhead es muy similar, además se puede observar que el algoritmo de FxT siempre tiene más overhead al requerir realizar la operación de transponer la matriz y aunque aumenta el overhead, este algoritmo disminuye el tiempo de ejecución total.

6. Conclusiones y recomendaciones

- FxT es superior a FxC, y la ventaja crece con el tamaño de la matriz. En matrices pequeñas ambos algoritmos tardan prácticamente lo mismo, porque el tiempo lo domina el overhead. A medida que la matriz crece, FxT se vuelve cada vez más rápido.
- Distribuir el trabajo solo es rentable para tamaños de matrices grandes
- El overhead es muy grande en las matrices pequeñas
- El algoritmo de FxT genera una ganancia en rendimiento en matrices grandes pero en matrices pequeñas genera grandes tiempos de overhead
- Que convenga el clúster o la ejecución local depende del algoritmo y del tamaño ya que como se analizo Con matrices pequeñas, la ejecución local siempre fue más rápida, porque los mensajes viajan por memoria compartida y no por la red.

Referencias

[0] Message Passing Interface Forum, MPI: A Message-Passing Interface Standard, Version 4.1, Nov. 2023. [En línea]. Disponible en: https://www.mpi-forum.org/docs/ [Accedido: 08-oct-2026].

[1] Y. Oyama, S. Sasaki, S. Ishiguro, R. Matsumiya, J. Murakami, y O. Tatebe, "Reduction of Operating System Jitter Caused by Page Reclaim," en ROSS '14, Múnich, Alemania, 2014, ACM. Disponible en: http://dx.doi.org/10.1145/2612262.2612270 [Accedido: 05-oct-2026].

[2] Rafael Pérez Laserna, " La Ley de los Grandes Números," GeoGebra. [En línea]. Disponible en: https://www.geogebra.org/m/edy5fexu . [Accedido: 05-oct-2026].

[3]Weber State University, "Row-Major Order," CS 1410 Textbook. [En línea]. Disponible en: https://icarus.cs.weber.edu/~dab/cs1410/textbook/7.Arrays/row_major.html. [Accedido: 05-oct-2026].

[4] “Hilo en computación: ¿Qué es? | Lenovo México”. Offizielle Website von Lenovo | Notebooks, Desktop-PCs, Tablets, Monitore | Lenovo DE. Accedido el 30 de abril de 2026. [En línea]. Disponible: https://www.lenovo.com/mx/es/glosario/hilo/

[5] Wikipedia, "Row- and column-major order," Wikipedia, The Free Encyclopedia. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Row-_and_column-major_order. [Accedido: 05-oct-2026].

[6] W. Stallings, Sistemas operativos: Aspectos internos y principios de diseño, 7ma ed. Pearson Educación, 2012, cap. 3 - 4. [Accedido: 05-oct-2026].

[7] M. E. Rodríguez García, "Principios cuantitativos del diseño de computadoras," Universidad Tecnológica de la Mixteca, 2009. [En línea]. Disponible en: https://www.utm.mx/~merg/AC/2009/2.7principios_cuantitativos.html. [Accedido: 05-oct-2026].

[8] EcuRed, "Ley de Amdahl," EcuRed. [En línea]. Disponible en: https://www.ecured.cu/Ley_de_Amdahl. [Accedido: 05-oct-2026].

[9] Cornell Virtual Workshop > MPI Point-to-Point > Point-to-Point Communication > Standard Send. (2026). Cornell.Edu. https://cvw.cac.cornell.edu/mpip2p/intro/standard-send[Accedido: 05-oct-2026]

[10] Chiba, T., Endo, T., & Matsuoka, S. (2007). High-Performance MPI Broadcast Algorithm for Grid Environments Utilizing Multi-lane NICs [Technical research paper].[Accedido: 05-oct-2026].

[11] **Silberschatz, A., Galvin, P. B., & Gagne, G. (2018).** *Operating System Concepts* (10th ed.). Wiley. *(Capítulo sobre CPU Scheduling y Cache Affinity / NUMA)*.[Accedido: 05-oct-2026].

[12]Cornell Virtual Workshop > MPI Point-to-Point > Point-to-Point Communication > Standard Send. (2026). Cornell.Edu. https://cvw.cac.cornell.edu/mpip2p/intro/standard-send.[Accedido: 05-oct-2026].

[13] Open MPI Project, Open MPI 5.0.x Documentation (paginas de MPI_Init, MPI_Comm_size, MPI_Comm_rank, MPI_Bcast, MPI_Send, MPI_Recv, MPI_Wtime, mpirun y transportes de red TCP y memoria compartida). [En línea]. Disponible en: https://docs.open-mpi.org/en/v5.0.x/ [Accedido: 13-oct-2026].

[14] W. Kendall, "MPI Send and Receive" y "MPI Broadcast and Collective Communication," MPI Tutorial. [En linea]. Disponible en: https://mpitutorial.com/tutorials/ [Accedido: 13-oct-2026].

[15] A. Grama, A. Gupta, G. Karypis y V. Kumar, Introduction to Parallel Computing, 2nd ed. Harlow, Reino Unido: Addison-Wesley, 2003, cap. 5 (métricas de rendimiento: overhead, speedup y eficiencia).

[16] J. L. Gustafson, "Reevaluating Amdahl's Law," Communications of the ACM, vol. 31, no. 5, pp. 532-533, 1988, doi: 10.1145/42411.42415.

[17] The Linux man-pages project, "gettimeofday(2) -Linux manual page," man7.org. [En línea]. Disponible en: https://man7.org/linux/man-pages/man2/gettimeofday.2.html [Accedido: 13-oct-2026].

[18]“Matrix Multiplication Background User’s Guide - NVIDIA Docs,” *NVIDIA Docs*, 2020. https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html

[19]A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing, Second Edition*. Pearson Education, 2003.